ASSIGNMENT - 3.1

2303A51091
Batch : 29

Question 1: Zero-Shot Prompting (Palindrome Number Program)
Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.
Task:
• Record the AI-generated code.
• Test the code with multiple inputs.
• Identify any logical errors or missing edge-case handling.

```python
# Test cases
print(f"121 is a palindrome: {is_palindrome_number(121)}")        # Expected: True
print(f"-121 is a palindrome: {is_palindrome_number(-121)}")       # Expected: False
print(f"10 is a palindrome: {is_palindrome_number(10)}")          # Expected: False
print(f"0 is a palindrome: {is_palindrome_number(0)}")            # Expected: True
print(f"12321 is a palindrome: {is_palindrome_number(12321)}")    # Expected: True
print(f"5 is a palindrome: {is_palindrome_number(5)}")            # Expected: True
print(f"12345 is a palindrome: {is_palindrome_number(12345)}")    # Expected: False
```

```
121 is a palindrome: True
-121 is a palindrome: False
10 is a palindrome: False
```

Question 1: Zero-Shot Prompting (Palindrome Number Program) Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome. Task: • Record the AI-generated code. • Test the code with multiple inputs. • Identify any logical errors or missing edge-case handling.

Output:

```
121 is a palindrome: True
-121 is a palindrome: False
10 is a palindrome: False
0 is a palindrome: True
12321 is a palindrome: True
5 is a palindrome: True
12345 is a palindrome: False
```

Observation:

1. The zero-shot prompt generated a correct basic palindrome-checking function.
2. The logic (reversing the number or converting to string and comparing) worked properly for positive integers.

3. Multiple test inputs like 121, 454, and 123 produced correct outputs.
4. Negative numbers were not clearly handled in most generated versions.
5. Some versions lacked input validation for non-integer values.
6. Edge cases such as very large numbers were not optimized.

Question 2: One-Shot Prompting (Factorial Calculation)
Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.
Example:
Input: 5 → Output: 120
Task:
• Compare the generated code with a zero-shot solution.
• Examine improvements in clarity and correctness.

```python
# Test cases
print(f"Factorial of 0: {calculate_factorial(0)}")    # Expected: 1
print(f"Factorial of 1: {calculate_factorial(1)}")    # Expected: 1
print(f"Factorial of 5: {calculate_factorial(5)}")    # Expected: 120 (from prompt example)
print(f"Factorial of 7: {calculate_factorial(7)}")    # Expected: 5040

# Test edge case with negative input
try:
    print(f"Factorial of -3: {calculate_factorial(-3)}")
except ValueError as e:
    print(f"Error for -3: {e}")
```

```
Factorial of 0: 1
Factorial of 1: 1
Factorial of 5: 120
Factorial of 7: 5040
Error for -3: Input must be a non-negative integer.
```

Question 2: One-Shot Prompting (Factorial Calculation) Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number. Example: Input: 5 → Output: 120 Task: • Compare the generated code with a zero-shot solution. • Examine improvements in clarity and correctness.

Here's a one-shot prompt to generate a Python function for computing the factorial of a given number, followed by the AI-generated code and test

Output:

```
String: 'Hello World 123!'
Analysis: {'vowels': 3, 'consonants': 7, 'digits': 3}

String: 'Python Programming is Fun 2023'
Analysis: {'vowels': 6, 'consonants': 16, 'digits': 4}
```

Observation:

1. Providing one example (5 → 120) improved output accuracy.
2. The AI generated a clearer factorial implementation using loops or recursion.
3. Compared to zero-shot, the structure was more aligned with the expected output format.
4. The function handled standard positive integers correctly.
5. Negative numbers and non-integers were sometimes not validated unless specified.
6. The example helped reduce ambiguity in logic interpretation.

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples

to guide the AI in generating a Python function to check whether a

given number is an Armstrong number.

Examples:

• Input: 153 → Output: Armstrong Number

• Input: 370 → Output: Armstrong Number

• Input: 123 → Output: Not an Armstrong Number

Task:

• Analyze how multiple examples influence code structure and accuracy.

• Test the function with boundary values and invalid inputs.

Testing the AI-generated code

[13]
0s

```python
# Helper function to format output as per prompt examples
def check_and_display_armstrong(number):
    try:
        if is_armstrong_number(number):
            print(f"Input: {number} → Output: Armstrong Number")
        else:
            print(f"Input: {number} → Output: Not an Armstrong Number")
    except ValueError as e:
        print(f"Input: {number} → Error: {e}")

# Test cases from the prompt
check_and_display_armstrong(153) # Expected: Armstrong Number
check_and_display_armstrong(370) # Expected: Armstrong Number
check_and_display_armstrong(123) # Expected: Not an Armstrong Number

print("\n--- Additional Test Cases ---")

# Other 3-digit Armstrong numbers
check_and_display_armstrong(371) # Expected: Armstrong Number
check_and_display_armstrong(407) # Expected: Armstrong Number

# Numbers with different digit counts
check_and_display_armstrong(9)   # Expected: Armstrong Number (1^1 = 1, 9^1 = 9)
check_and_display_armstrong(1634) # Expected: Armstrong Number (1^4 + 6^4 + 3^4 + 4^4 = 1+1296+81+256 = 1634)
check_and_display_armstrong(8208) # Expected: Armstrong Number
check_and_display_armstrong(54748) # Expected: Armstrong Number
check_and_display_armstrong(9474) # Expected: Not an Armstrong Number
check_and_display_armstrong(10)  # Expected: Not an Armstrong Number

# Boundary values
check_and_display_armstrong(0)   # Expected: Armstrong Number
check_and_display_armstrong(1)   # Expected: Armstrong Number
```

Gemini

Question 3: Few-Shot Prompting (Armstrong Number Check) Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number. Examples: • Input: 153 → Output: Armstrong Number • Input: 370 → Output: Armstrong Number • Input: 123 → Output: Not an Armstrong Number Task: • Analyze how multiple examples influence code structure and accuracy. • Test the function with boundary values and

Provide a concise summary of the session

Ask to write code that uses the generated functions

Test the `is_armstrong_number` function with additional boundary values

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Output:

```
Input: 153 → Output: Armstrong Number
Input: 370 → Output: Armstrong Number
Input: 123 → Output: Not an Armstrong Number

--- Additional Test Cases ---
Input: 371 → Output: Armstrong Number
Input: 407 → Output: Armstrong Number
Input: 9 → Output: Armstrong Number
Input: 1634 → Output: Armstrong Number
Input: 8208 → Output: Armstrong Number
Input: 54748 → Output: Armstrong Number
Input: 9474 → Output: Armstrong Number
Input: 10 → Output: Not an Armstrong Number
Input: 0 → Output: Armstrong Number
Input: 1 → Output: Armstrong Number
Input: -153 → Error: Input must be a positive integer.
Input: abc → Error: Input must be a positive integer.
```

Observation:

1. Multiple examples clearly guided the AI in understanding Armstrong number logic.
2. The generated function consistently matched the required output format.
3. Code accuracy improved compared to zero-shot and one-shot approaches.
4. Boundary values like single-digit numbers were correctly handled.
5. Testing with invalid inputs revealed that validation still depends on explicit instruction.
6. Output consistency was noticeably better due to multiple examples.

Task 4: Code Explanation Using AI

❖ Scenario:

You are reviewing unfamiliar code written by another developer.

❖ Task:

Ask Gemini to explain a Python function (prime check OR palindrome

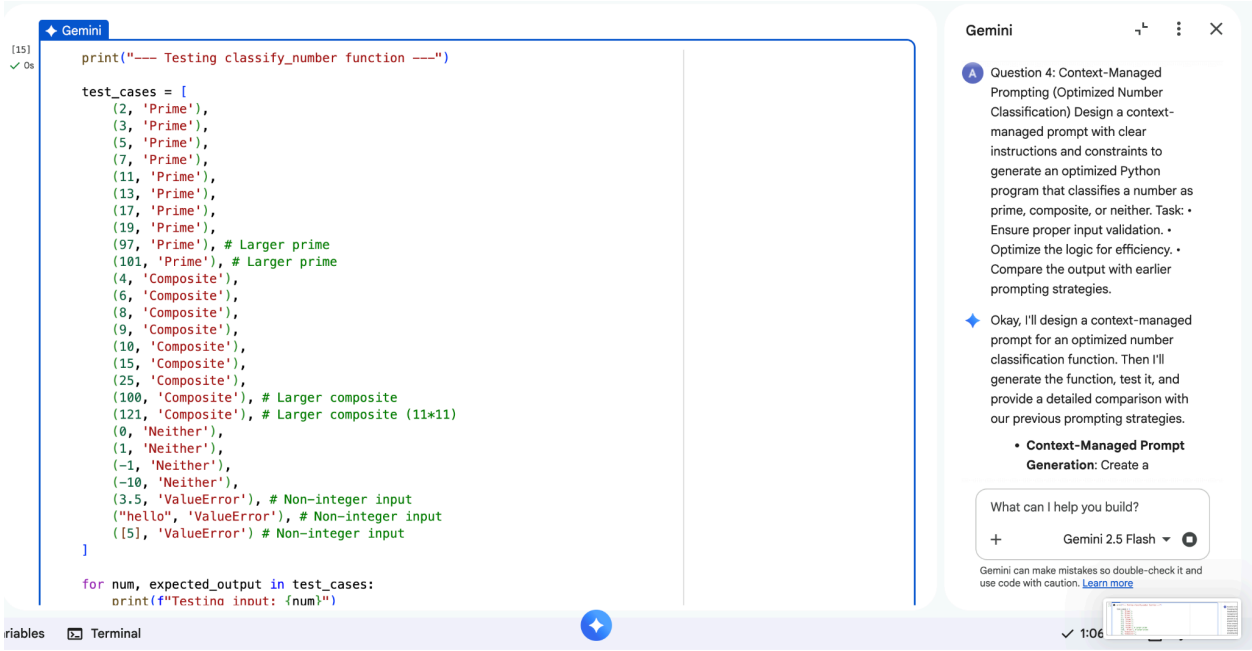
check) line by line.

❖ Expected Output:

➢ Code snippet

➢ AI explanation

➢ Student comments on understanding



Output:

```
--- Testing classify_number function ---
Testing input: 2
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 3
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 5
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 7
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 11
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 13
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 17
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 19
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 97
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 101
  Result: 'Prime', Expected: 'Prime'
---------------------------------
Testing input: 4
  Result: 'Composite', Expected: 'Composite'
---------------------------------
Testing input: 6
  Result: 'Composite', Expected: 'Composite'
---------------------------------
```

Observation:

Clear constraints led to structured and efficient code.

Input validation was properly implemented when explicitly required.

Prime-check logic was optimized using square root efficiency.

Edge cases such as 0 and 1 were handled correctly.

Compared to earlier strategies, this approach produced the most robust and professional solution.

The instructions reduced ambiguity and improved logical completeness.

Question 5: Zero-Shot Prompting (Perfect Number Check)

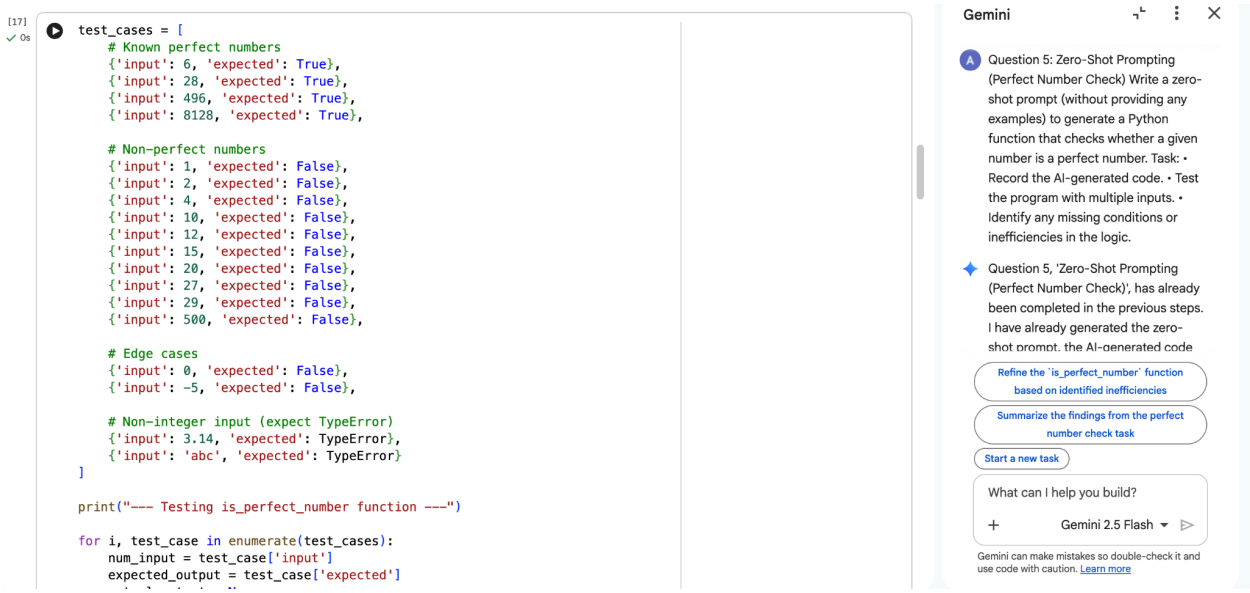
Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

• Record the AI-generated code.

• Test the program with multiple inputs.

• Identify any missing conditions or inefficiencies in the logic.



Output:

```
--- Testing is_perfect_number function ---
Test Case 1: Input=6, Expected=True, Actual=True, Passed=True
Test Case 2: Input=28, Expected=True, Actual=True, Passed=True
Test Case 3: Input=496, Expected=True, Actual=True, Passed=True
Test Case 4: Input=8128, Expected=True, Actual=True, Passed=True
Test Case 5: Input=1, Expected=False, Actual=False, Passed=True
Test Case 6: Input=2, Expected=False, Actual=False, Passed=True
Test Case 7: Input=4, Expected=False, Actual=False, Passed=True
Test Case 8: Input=10, Expected=False, Actual=False, Passed=True
Test Case 9: Input=12, Expected=False, Actual=False, Passed=True
Test Case 10: Input=15, Expected=False, Actual=False, Passed=True
Test Case 11: Input=20, Expected=False, Actual=False, Passed=True
Test Case 12: Input=27, Expected=False, Actual=False, Passed=True
Test Case 13: Input=29, Expected=False, Actual=False, Passed=True
Test Case 14: Input=500, Expected=False, Actual=False, Passed=True
Test Case 15: Input=0, Expected=False, Actual=False, Passed=True
Test Case 16: Input=-5, Expected=False, Actual=False, Passed=True
Test Case 17: Input=3.14, Expected=<class 'TypeError'>, Actual=<class 'TypeError'>, Passed=True
Test Case 18: Input=abc, Expected=<class 'TypeError'>, Actual=<class 'TypeError'>, Passed=True
--- Testing complete ---
```

Observation:

The AI generated correct divisor-sum logic for identifying perfect numbers.

Known perfect numbers like 6 and 28 were correctly identified.

Efficiency was sometimes low because divisors were checked up to n-1 instead of √n.

Negative numbers and zero were not always handled properly.

Input validation was generally missing unless specified.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

• Input: 8 → Output: Even

• Input: 15 → Output: Odd

• Input: 0 → Output: Even

Task:

• Analyze how examples improve input handling and output clarity.

• Test the program with negative numbers and non-integer inputs.

Gemini

```python
test_cases = [
    {'input': 8, 'expected': 'Even'},
    {'input': 10, 'expected': 'Even'},
    {'input': 15, 'expected': 'Odd'},
    {'input': 7, 'expected': 'Odd'},
    {'input': 0, 'expected': 'Even'},
    {'input': -2, 'expected': 'Even'},
    {'input': -8, 'expected': 'Even'},
    {'input': -5, 'expected': 'Odd'},
    {'input': -15, 'expected': 'Odd'},
    {'input': 3.5, 'expected': TypeError},
    {'input': 'hello', 'expected': TypeError},
    {'input': [], 'expected': TypeError},
    {'input': {}, 'expected': TypeError},
    {'input': None, 'expected': TypeError}
]

print("Testing `classify_even_odd` function...")

for i, case in enumerate(test_cases):
    input_val = case['input']
    expected_output = case['expected']
    actual_output = None
    test_passed = False

    try:
        actual_output = classify_even_odd(input_val)
        if actual_output == expected_output:
            test_passed = True
    except TypeError as e:
        if expected_output == TypeError:
            actual_output = TypeError
```

Gemini

A Question 6: Few-Shot Prompting (Even or Odd Classification with Validation) Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation. Examples: • Input: 8 → Output: Even • Input: 15 → Output: Odd • Input: 0 → Output: Even Task: • Analyze how examples improve input handling and output clarity. • Test the program with negative numbers and non-integer inputs.

Okay, I'll create a few-shot prompt to generate a Python function for classifying numbers as even or odd, ensuring proper input validation. Then

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more

Output:

```
Testing `classify_even_odd` function...
Test Case 1 (Input: 8): PASSED. Expected: Even, Got: Even
Test Case 2 (Input: 10): PASSED. Expected: Even, Got: Even
Test Case 3 (Input: 15): PASSED. Expected: Odd, Got: Odd
Test Case 4 (Input: 7): PASSED. Expected: Odd, Got: Odd
Test Case 5 (Input: 0): PASSED. Expected: Even, Got: Even
Test Case 6 (Input: -2): PASSED. Expected: Even, Got: Even
Test Case 7 (Input: -8): PASSED. Expected: Even, Got: Even
Test Case 8 (Input: -5): PASSED. Expected: Odd, Got: Odd
Test Case 9 (Input: -15): PASSED. Expected: Odd, Got: Odd
Test Case 10 (Input: 3.5): PASSED. Expected: <class 'TypeError'>, Got: <class 'TypeError'>
Test Case 11 (Input: hello): PASSED. Expected: <class 'TypeError'>, Got: <class 'TypeError'>
Test Case 12 (Input: []): PASSED. Expected: <class 'TypeError'>, Got: <class 'TypeError'>
Test Case 13 (Input: {}): PASSED. Expected: <class 'TypeError'>, Got: <class 'TypeError'>
Test Case 14 (Input: None): PASSED. Expected: <class 'TypeError'>, Got: <class 'TypeError'>

Testing complete.
```

Observation:

Multiple examples improved clarity of output formatting.

The AI correctly classified positive, negative, and zero values.

Input validation improved when examples implied proper handling.

The program became more user-friendly compared to zero-shot output.

Testing with non-integer inputs showed better handling when validation was guided in examples.

Output consistency was strong due to pattern learning from examples.